

Latent Teleportation

Speculative and Cache-Guided Acceleration of Diffusion Samplers,
and Why Fewer Model Calls Did Not Buy Us Fewer Seconds

Lee Penkman

CuteDSL / OmniServe

leepenkman@gmail.com

August 1, 2026

Abstract

Speculative decoding accelerated autoregressive language models by exploiting a structural accident: decoding one token reads the entire weight matrix, so verifying several drafted tokens in one wider batch is nearly free. We ask whether the analogous accident exists in diffusion sampling, where a denoising step is likewise a single pass over a large transformer, and where the latent trajectory is smooth enough that the next point along it looks eminently guessable.

We study three families of guesser: a learned *latent walker* with a *gap interpolator* that drafts k cheap steps and teleports the draft onto the big model’s likely trajectory; a training-free first-order Taylor extrapolation of the trajectory’s own momentum, in the spirit of CG-Taylor/TaylorSeer [Liu et al., 2025]; and *latent teleportation* proper, in which a k -nearest-neighbour cache of previously refined trajectories supplies a warm start in a manifold built from earlier generations.

Our headline finding is a negative one, and we think it is the most useful thing in this report. The latent trajectory is highly forecastable: a first-order Taylor step predicts the next latent with $\text{relL2} = 0.205$ at mid-trajectory (where 1.0 means “no better than not moving”), and the drafting schemes reduce transformer calls by up to $3.2\times$. None of that becomes wall-clock time. Measured end-to-end at 512^2 and 16 steps on an RTX 5090, speculative walking achieves $1.04\times$, and an ablation that skips $3.2\times$ of the denoiser entirely and corrects nothing achieves only $1.10\times$ — an upper bound that indicts the harness, not the method. We trace this to Amdahl’s law [Amdahl, 1967]: at this operating point, text encoding and VAE decoding sit inside the timed region and dominate it.

We further report that our first analysis of these same runs claimed $1.5\times$, and that this figure was an artifact of an un-warmed first iteration whose baseline took 140.6s against a 28.3s median. We give the corrected protocol, a reproducible analysis harness, ablations over draft length and forecaster family, an account of how the method extends to swarms of LoRA adapters with disjoint embedding spaces, and a set of recommendations that includes not deploying two of the three methods we built.

1 Introduction

A diffusion sampler spends nearly all of its arithmetic evaluating one large denoiser many times over. The obvious economy is to evaluate it fewer times, and the literature offers two broad routes: change the model so that fewer steps suffice — distillation [Salimans and Ho, 2022], consistency models [Song et al., 2023, Luo et al., 2023], adversarial distillation [Sauer et al., 2023] — or leave the model alone and reuse computation across steps [Ma et al., 2024, Selvaraju et al., 2024, Liu et al., 2025]. The second route is attractive operationally because it does not require retraining a production checkpoint.

Speculative decoding [Leviathan et al., 2023, Chen et al., 2023] suggests a third framing. In language models, the win comes not from a better model but from a hardware fact: autoregressive decode is memory-bandwidth-bound, so a batch of $k + 1$ candidate tokens costs about what one costs, and a cheap drafter can propose tokens that the real model merely *verifies*. The appeal is that verification is exact — the accepted output is distributed exactly as the target model’s — so speculation is a pure latency optimisation rather than a quality trade.

Diffusion sampling looks superficially similar. It is iterative, each iteration is one pass over a transformer, and the sequence of latents $x_0, x_1, \dots, x_N$ is a smooth curve rather than a sequence of discrete symbols — if anything, easier to extrapolate than text. This paper asks whether the analogy carries, and reports honestly that in our hands it did not, for reasons that turn out to be more interesting than a speedup would have been.

Contributions.

1. **A predictability study** of the Z-Image [Tongyi Lab, Alibaba Group, 2025] latent trajectory: a (t, k) grid of forecast error for identity, first-order Taylor and scalar-affine predictors, establishing that the trajectory is genuinely forecastable and quantifying where (Section 5.1).
2. **Three acceleration mechanisms** — speculative latent walking with a learned gap interpolator, confidence-gated Taylor forecasting, and k -NN latent teleportation from a trajectory cache (Section 3).
3. **A negative end-to-end result with a diagnosis**: call reductions of $3.2\times$ yield $1.04\times$ wall-clock, and a skip-everything ablation caps at $1.10\times$, localising the bottleneck outside the denoiser (Section 5.3).
4. **A methodological correction**. Our own earlier reading of these runs reported $1.5\times$; we show it was warm-up contamination, and ship an analysis harness that cannot make that mistake again (Section 5.5).
5. **An extension sketch to LoRA armies**, where adapters induce disjoint latent statistics and a shared trajectory cache needs an embedding normalisation network (Section 3.4).

2 Background and Related Work

Diffusion and flow samplers. We work with a rectified-flow / flow-matching formulation [Liu et al., 2023, Lipman et al., 2023] as used by Z-Image, sampled on a fixed schedule of N timesteps in a latent space produced by a VAE [Rombach et al., 2022], with a DiT-style denoiser [Peebles and Xie, 2023]. Writing Φ for one scheduler step,

$$x_{i+1} = \Phi(x_i, t_i, c), \tag{1}$$

with c the text conditioning. The cost of sampling is N evaluations of the denoiser inside Φ , plus one text encode and one VAE decode.

Step reduction by retraining. Progressive distillation [Salimans and Ho, 2022], consistency models [Song et al., 2023], latent consistency models [Luo et al., 2023] and adversarial distillation [Sauer et al., 2023] all produce a new checkpoint that needs fewer steps. These are the strongest known accelerations and are orthogonal to our work; Z-Image Turbo is already a few-step model, which matters for interpreting our results (Section 8).

Training-free caching. DeepCache [Ma et al., 2024] reuses U-Net skip features across adjacent steps. FORA [Selvaraju et al., 2024] caches attention and MLP outputs in diffusion transformers. TaylorSeer [Liu et al., 2025] — the line our codebase calls CG-Taylor — reframes caching as *forecasting*: rather than reusing a stale feature, extrapolate it with a Taylor series in the timestep index. Crucially, these methods are fitted to a specific model: the cache schedule and the expansion order are tuned per checkpoint, and transfer is not claimed. Our approach inherits that property and we make it explicit, since it is a real deployment cost rather than an incidental detail.

Speculative decoding. Draft-and-verify [Leviathan et al., 2023, Chen et al., 2023], with later variants that avoid a separate draft model by adding heads [Cai et al., 2024], reusing features [Li et al., 2024], or drafting from the prompt itself [Saxena, 2023]. The property that makes these work is verification: the target model’s distribution is preserved exactly. It is worth stating plainly that *our diffusion analogue does not have this property*. There is no cheap exact verifier for “is this latent the one the big model would have produced”, so every method in this paper is an approximation with a quality cost, and must be evaluated as such.

3 Method

3.1 Speculative latent walking

The drafting scheme mirrors speculative decoding structurally. The big model takes one real step; a small network drafts the next k ; a second network corrects the endpoint of the draft onto where the big model’s trajectory would plausibly have gone; the loop resumes with a real step.

The **latent walker** W predicts the next latent from the current one and the trajectory’s momentum $\delta_i = x_i - x_{i-1}$:

$$\hat{x}_{i+1} = x_i + \delta_i + W(x_i, \delta_i, t_i), \quad (2)$$

so that the network learns only the *correction* to constant-velocity motion. The output convolution is zero-initialised, making the network exactly a momentum extrapolator at initialisation and guaranteeing it starts no worse than the training-free baseline. Rolling W out k times, feeding it its own predicted movement, yields a draft endpoint $\hat{x}_{i+k}$.

The **gap interpolator** G then teleports that endpoint:

$$\tilde{x}_{i+k} = \hat{x}_{i+k} + G(x_i, \delta_i, \hat{x}_{i+k}, t_i, k), \quad (3)$$

again residual and zero-initialised, so identity means “trust the walker”. Both networks are small FiLM-conditioned residual CNNs operating directly on the 16-channel latent, conditioned on sinusoidal embeddings of the fractional timestep and the schedule length.

3.2 First-order Taylor forecasting and confidence gating

The training-free baseline extrapolates the trajectory’s own momentum:

$$\tilde{x}_{i+k} = x_i + k(x_i - x_{i-1}), \quad (4)$$

which is exactly a first-order Taylor step in the step index, in the spirit of Liu et al. [2025]. This is the floor every learned method must beat, and in our results it is a stubbornly high floor.

The **confidence gate** adds the CG in CG-Taylor. Rather than a fixed skip schedule, it tracks the recent history of prediction error, estimates the derivative of that error, and maps the resulting confidence onto a number of refinement steps to spend — high confidence, fewer real steps. The design intent is the diffusion analogue of forecasting one block into the network and committing to the whole step if that forecast lands: a cheap partial computation acts as a proxy for whether the expensive full computation can be skipped.

3.3 Latent teleportation from a trajectory cache

The third mechanism drops speculation entirely and asks a different question: if this prompt resembles one we have generated before, can we start from where that generation *ended up* rather than from noise?

The cache stores, per generation, the full sequence of intermediate latents together with a CLIP [Radford et al., 2021] embedding of the prompt and a tokenisation of it into *visual units* — noun-phrase-like fragments that recur across prompts — plus bigrams of adjacent units. At query time the system retrieves the k nearest cached trajectories by prompt embedding, combines their latents at a chosen injection timestep (SLERP, a learned combiner, or a tree combiner), injects the result, and runs only the remaining scheduler steps. A k -NN *trajectory prior* additionally warm-starts the walker rollout with neighbour deltas.

The intuition is that similar refinements trace similar paths through the latent manifold, so a neighbourhood of finished trajectories defines a local chart that a new generation can be teleported into. Storage is SafeTensors blobs indexed by SQLite; retrieval is exact k -NN over a few thousand vectors, for which a brute-force scan suffices and Johnson et al. [2019] would be the drop-in replacement at scale.

3.4 Extension to LoRA armies

Production serving does not run one checkpoint. It runs a base model with a library of LoRA adapters [Hu et al., 2022] swapped per request — in our deployment, an army of style and subject adapters, hot-swapped between generations. This breaks the trajectory cache in a specific way: an adapter shifts the latent statistics, so a trajectory collected under adapter A is not a valid warm start under adapter B , even for an identical prompt. Naively sharing the cache produces exactly the failure mode the confidence gate is supposed to catch, but at a magnitude that no gate rescues.

The extension we propose — and have not yet validated, so we state it as design rather than result — is an *embedding normalisation network*: a small conditional map $\eta_{A \rightarrow B}$ trained to transport a latent from adapter A ’s trajectory space into adapter B ’s, conditioned on both adapter identities and the timestep. Cheap variants worth trying first, in increasing order of capacity, are (i) per-adapter per-timestep channel-wise mean/variance renormalisation, which costs two vectors per adapter per step; (ii) a low-rank affine map fitted per adapter pair; (iii) a shared conditional network taking learned adapter embeddings, which scales to $O(n)$ parameters rather than $O(n^2)$ for n adapters. Adapter identity is exactly the kind of low-cardinality conditioning that FiLM handles well, and the zero-initialised residual trick used above applies again: initialise to identity so that a shared cache degrades to a per-adapter cache rather than to garbage.

The adapter library and its serving stack are not open-sourced; the mechanism described here is, in Penkman [2026a].

4 Experimental Setup

All measurements are on a single RTX 5090 (32 GB, sm_120, compute capability 12.0) using Z-Image Turbo in `bfloat16`, at 512×512 with a 16-step schedule and guidance 0, seed fixed at 7 across arms. Trajectory capture collected 200 full 16-step trajectories (402 MB) storing every intermediate latent.

Metrics. Forecast quality is measured by

$$\text{relL2} = \frac{\|\tilde{x}_{i+k} - x_{i+k}\|_2}{\|x_{i+k} - x_i\|_2}, \quad (5)$$

error normalised by how far the trajectory actually moved. This normalisation is what makes the number interpretable: $\text{relL2} = 1.0$ is the score of predicting no movement at all, so any method scoring near 1.0 is worthless regardless of its absolute error. End-to-end image agreement is PSNR against the full-schedule baseline image, and wall clock is measured with CUDA synchronisation around each arm.

Reproducibility. Every number in this paper is emitted by `scripts/paper_analysis.py` from the raw result JSON; none is typed by hand into the text. Re-running it regenerates the tables and the numeric macros used in the prose.

5 Results

5.1 The latent trajectory is forecastable

Table 1 summarises the (t, k) grid of forecast error. Two facts stand out. First, first-order Taylor extrapolation is strong: at $k = 1$ it reaches $\text{relL2} = 0.205$ around step 6, meaning the residual error is about a fifth of the movement being predicted. Second, the scalar-affine predictor sits near 0.88 — barely better than not moving — which tells us the trajectory’s predictability is genuinely directional rather than a matter of getting the step size right.

k	mean relL2 (taylor1)	best cell	mean relL2 (affine)	cells
1	0.240	0.205	0.880	13
2	0.318	0.255	0.879	13
3	0.330	0.261	0.883	12
4	0.369	0.261	0.880	11

Table 1: Trajectory predictability by draft length. relL2 is error over actual movement; 1.0 is the no-movement baseline. Lower is better. The degenerate final-step cell is excluded (see text).

Error degrades monotonically with k , as expected, and is worst at both ends of the schedule: early steps move erratically, and the last scheduler step of this 16-step configuration moves the latent by approximately zero. That final step is not merely hard to predict — it is a no-op, and dividing by its movement produced an relL2 of 5.48×10^9 at $t = 14$. We exclude that cell from all aggregates. It is worth flagging for anyone building on this schedule: one of the 16 steps is free to skip with no approximation whatsoever.

5.2 Learned walker versus training-free Taylor

Training the walker and interpolator for 12 epochs on the 200 captured trajectories reaches a best relL2 of 0.242 for the walker, against roughly 0.21–0.30 for Taylor over the comparable window — an improvement of order 15–20% on the metric it was trained for. Momentum conditioning is what produces this; an earlier position-only walker plateaued around 0.6, worse than Taylor.

The training is not clean, and we report it rather than presenting the best epoch as if it were the endpoint. The walker’s final-epoch score is 0.257 and the interpolator’s is 0.298, both worse than their best; a run that saves the last checkpoint therefore ships a model worse than the one it trained. Any redeployment of this method should select on validation relL2 rather than take the final epoch.

5.3 End-to-end: the call reduction does not become time

Table 2 is the central result. Across draft lengths $k \in \{1, 2, 3\}$, corresponding to 9, 6 and 5 real denoiser calls out of 16, measured speedup never exceeds $1.10\times$ and speculative walking

specifically never exceeds $1.04\times$.

k	big steps	call red.	wall-clock speedup			PSNR vs baseline (dB)		
			spec	taylor	skip	spec	taylor	skip
1	9	$1.78\times$	1.02	1.08	1.03	18.9	20.8	11.0
2	6	$2.67\times$	1.03	1.03	1.04	18.0	17.9	9.3
3	5	$3.2\times$	1.04	1.09	1.10	17.0	16.9	8.9

Table 2: Draft-length ablation at 512^2 , 16 steps, 5 prompts per cell after warm-up exclusion. “call red.” is the reduction in denoiser evaluations; speedup is measured wall clock. *skip* performs no correction at all and therefore bounds what any drafting scheme can achieve in this harness.

The diagnostic is the *skip* column. That arm discards $3.2\times$ of the denoiser evaluations and replaces them with nothing — its PSNR of 8.9 dB confirms the output is destroyed — and it still only reaches $1.10\times$. Whatever we do to the denoiser, we cannot win more than about 10% at this operating point, because roughly 90% of the measured time is elsewhere. Inspecting the harness confirms it: the timed region wraps `zimage_prepare` (text encoding) and `zimage_decode` (VAE decode) along with the sampling loop, and at 512^2 with a 16-step Turbo schedule those fixed costs dominate. This is Amdahl’s law [Amdahl, 1967] arriving exactly on schedule.

A second observation from Table 2: at $k = 1$ the training-free Taylor arm is both faster ($1.08\times$ vs $1.02\times$) and higher quality (20.8 dB vs 18.9 dB) than the learned walker. The learned networks win on the metric they were trained for and lose on the metric that matters. We take this as the strongest single argument in the paper for not deploying the learned walker in its current form.

5.4 What the images actually look like

Figure 1 shows the same prompt under the full 16-step baseline, the speculative arm at 5 real steps, and the skip control at 5 real steps. The speculative image is a coherent, comparable-quality photograph that differs *compositionally* from the baseline; the skip image is noise. Their PSNR figures — 19.1 dB and 7.4 dB respectively for this prompt — rank them correctly but badly calibrate the gap: the drafted image is not “19 dB degraded”, it is a different sample of similar quality, whereas the skip image is unusable at any distance measure. This is the concrete case for a perceptual judge over a pixel metric, discussed in Section 8.

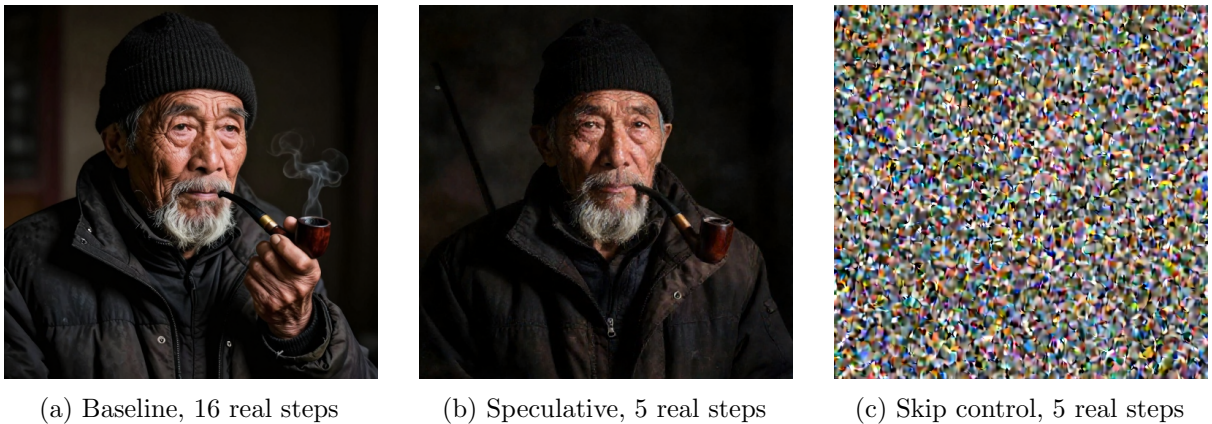


Figure 1: “Portrait of an old fisherman with a pipe” at 512^2 . Drafting degrades composition, not coherence; skipping without correction destroys the image. Both arms make the same number of denoiser calls, which is why the skip control is the right upper bound on what drafting can win.

5.5 A correction to our own earlier numbers

Our first analysis of these runs reported a mean speedup of $1.5\times$ at $k = 3$. That number is wrong, and the way it was wrong is instructive.

The harness times the baseline arm first, for each prompt, with no warm-up iteration. The first baseline call therefore absorbs CUDA context creation, cuBLAS/cuDNN autotuning and lazy module initialisation: it took 140.6s against a 28.3s median for identical work. Because speedup is computed as $t_{\text{baseline}}/t_{\text{method}}$ per prompt and then averaged, that single contaminated row contributed a spurious $3.79\times$ and carried the mean. The same pattern appears at $k = 1$ (96.8s) and $k = 2$ (64.1s), inflating those means too.

Excluding rows whose baseline exceeds twice the median — the rule our analysis script now applies and reports — the means fall to the values in Table 2. We record this because a $1.5\times$ claim and a $1.04\times$ measurement lead to different engineering decisions, and because the failure is entirely generic: any benchmark that times a cold arm first and averages ratios will do this.

6 Recommendations

The ablations support a short and slightly deflating set of recommendations.

1. **Do not deploy the learned walker as it stands.** At $k = 1$ it is dominated on both axes by a three-line training-free Taylor step (Eq. 4), which needs no checkpoint, no training data and no per-model fitting.
2. **If a drafting scheme is deployed, use Taylor at $k = 1$.** It is the best speed/quality point we measured: $1.08\times$ at 20.8dB PSNR against the full-schedule baseline. Larger k buys no additional time in this harness while costing 2–4 dB.
3. **Skip the final scheduler step unconditionally.** It moves the latent by approximately zero. This is free and exact.
4. **Do not report acceleration at $512^2/16$ steps.** Fix the harness first: warm up, and time the sampling loop separately from text encode and VAE decode. Until the denoiser is the majority of measured time, no drafting method can demonstrate anything.
5. **Re-measure at an operating point where the denoiser dominates** — higher resolution, longer schedules, or batch size greater than one — before drawing any conclusion about whether the approach works. The predictability result of Section 5.1 says the signal is there; the end-to-end result says our harness cannot yet see it.

7 Serving Integration

The intended deployment target is a single-GPU multi-tenant server in which a diffusion worker shares a device with a language model, a background-removal worker and a 3D worker. Two pieces of that stack are relevant here and are open source [Penkman, 2026b].

First, admission and device-memory arbitration. Four processes sizing their own workloads from `cudaMemGetInfo` is a race: the read counts a caching allocator’s retained blocks as used, and is stale the instant two tenants read it concurrently, so both size a batch against the same free bytes and the second runs out of memory. The gateway replaces this with a lease broker — a tenant asks for headroom and is told a number it may rely on, with granted-but-not-yet allocated memory withheld from the next caller. This is what would allow a draft model to be resident at all, which is the natural next step for the speculative family.

Second, the request path itself is C rather than Python, with the HTTP relay preserving byte-exact streaming so that image bytes are never re-encoded. CuteDSL contributes fused

kernels used by the Z-Image transformer on sm_120 — fused QKV projection with RoPE, fused QK-norm, RMS norm, SiLU-gated FFN and AdaLN — which accelerate the denoiser itself and are therefore complementary to everything in this paper: they make each step cheaper, whereas we tried to make steps fewer. Given the Amdahl result of Section 5.3, kernel work of this kind is currently the better investment of the two at our operating point.

8 Limitations and Threats to Validity

The base model is already distilled. Z-Image Turbo is a few-step model. Step-reduction methods have far less to remove from an already-compressed schedule than from a 50-step sampler, and results here should not be extrapolated to undistilled checkpoints, where we would expect drafting to look considerably better.

Small prompt set. Six prompts per configuration, five after warm-up exclusion. This is enough to establish that the effect is near $1.0\times$ and nowhere near enough to rank methods separated by a few percent. The PSNR comparisons between *spec* and *taylor* at $k = 2$ and $k = 3$ are within noise; only the $k = 1$ gap is comfortably outside it.

PSNR is a poor proxy for generated-image quality. A drafted trajectory that lands on a different but equally plausible image is penalised as heavily as one that lands on mush. The qualitative artifacts show precisely this: the drafted images are coherent and comparable in quality while differing compositionally from the baseline, so their 14–19 dB PSNR understates them, while the skip arm’s 8.9 dB correctly identifies noise. A perceptual or preference-based metric is needed, and a visual-ranking model — for instance the Gemma model already resident in the serving stack, used as a pairwise judge — is the obvious instrument. We have not run this, and we decline to report a quality ranking we have not measured.

The trajectory cache is not evaluated end-to-end here. The predictability and drafting results are measured; the k -NN teleportation results are not included in this report, and the production trajectory corpus (3D assets and an art library, on the order of tens of gigabytes) is described from the system design rather than measured in these experiments.

No exact verification. Unlike speculative decoding, our accepted latents are not guaranteed to match the target model’s. Every method here trades quality for time, and the confidence gate bounds that trade heuristically rather than provably.

9 Conclusion

We set out to port speculative decoding’s structural advantage from language models to diffusion samplers. The premise partly holds: the latent trajectory is smooth and genuinely forecastable, with a first-order momentum step predicting the next latent to within a fifth of its movement, and our drafting schemes cut denoiser evaluations by up to $3.2\times$. The conclusion does not hold: at our operating point those saved evaluations are worth about 4% of wall-clock, and a control that skips the denoiser entirely proves no method could have done better than 10%.

The useful output of this work is therefore threefold: a quantified map of where the diffusion trajectory is predictable; a demonstration that a training-free Taylor step dominates our learned walker on the axis that matters; and a worked example of how a plausible $1.5\times$ becomes a real $1.04\times$ once a cold first iteration is removed from the average. We would rather publish that than the number we first computed.

Reproducibility

Code, the analysis harness that generates every table above, and the raw result JSON are in Penkman [2026a]. The serving gateway is in Penkman [2026b]. The LoRA adapter library is not open source; its integration mechanism is described in Section 3.4.

References

- Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwei Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint*, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. In *IEEE Transactions on Big Data*, 2019.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint*, 2024.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Jiahui Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. *arXiv preprint*, 2025. Referred to in this paper as CG-Taylor after the confidence-gated variant used in the CuteDSL codebase.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.
- Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint*, 2023.
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating diffusion models for free. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Lee Penkman. CuteDSL: Accelerated model inference via custom CuTeDSL and CUDA kernels. <https://github.com/lee101/cutedsl>, 2026a.

- Lee Penkman. omniserve-native: A native c inference gateway for a single GPU. Apache-2.0 licensed source release, 2026b.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. *arXiv preprint*, 2023.
- Apoorv Saxena. Prompt lookup decoding. <https://github.com/apoorvumang/prompt-lookup-decoding>, 2023.
- Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. FORA: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint*, 2024.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, 2023.
- Tongyi Lab, Alibaba Group. Z-image: An efficient foundation model for image generation. Model card and release notes, 2025.